

Programowanie Funkcyjne 2025

Lista zadań nr 10

Na zajęcia 22 i 23 stycznia 2026

Ta lista zadań dotyczy handlerów efektów algebraicznych. Programy należy napisać w języku Fram. Interpreter DBL języka Fram można pobrać i zainstalować wykonując poniższe polecenia.

```
$ git clone https://github.com/fram-lang/dbl
$ cd dbl
$ dune build
$ dune install
```

O tym jak używać języka można znaleźć w przykładach dołączonych do repozytorium (oraz przeglądając katalog z testami). Strzępy dokumentacji znajdują się na <https://fram-lang.org>.

Zadanie 1 (2p). Obliczenia ze stanem możemy wyrazić przy pomocy poniższego efektu ST z dwiema operacjami.

```
pub data ST X E =
  { get : Unit ->[E] X
  , put : X    ->[E] Unit
  }
```

Znaczenie operacji get i put możemy nadać później, uruchamiając takie obliczenie w odpowiednim handlerze, który dostarczy efektodzieja typu ST. Podobne oddzielenie definicji obliczenia i interpretacji samego efektu możemy uzyskać przy pomocy wolnej monady.

```
pub data rec FreeST X R A =
  | Return of A
  | Get    of (X ->[R] FreeST X R A)
  | Put    of X, (Unit ->[R] FreeST X R A)
```

We Framie taką wolną monadę możemy dodatkowo sparametryzować pozostałymi efektami (R), które mają miejsce wewnątrz takiego obliczenia ze stanem.

Pokaż, że obliczenie używające efektu ST X E można zinterpretować jako wolną monadę, tzn. napisz handler pierwszej kategorii, który będzie produkował wolną monadę. Twoja definicja powinna mieć następujący kształt.

```
pub let hFreeST =
  handler ST
  { effect get () = ...
  , effect put st = ...
  }
  return x => ..
end
```

Zadanie 2 (2p). Pokaż, że wolną monadę FreeST można uruchomić mając do dyspozycji efekt ST. Możesz to zrobić, definiując (rekurencyjną) metodę typu ST

```
parameter ~st : ST _ _
pub method rec run (self : FreeST _ _ _) =
  match self with
```

```
...  
end
```

Zadanie 3 (4p). Niektóre języki programowania, np. Python oferują tak zwane generatory, które pozwalają na wygodną implementację iteratorów. We Framie takiej dodatkowej funkcjonalności nie potrzebujemy, bo generatory można naturalnie wyrazić jako funkcje, które korzystają z pewnego niejawnego parametru `~yield`, który może mieć efekt obliczeniowy.

```
pub data Generator X =  
  Generator of { run : {E, ~yield : X ->[E] Unit} -> Unit ->[E] Unit }
```

Takie generatory można tworzyć na przykład w następujący sposób.

```
let example = Generator  
{ run () =  
  ~yield 1;  
  ~yield 2;  
  ~yield 3  
}
```

Zaimplementuj metody `iter` oraz `toList` dla typu `Generator`

```
pub method iter (Generator { run }) f = ...  
  
pub method toList (Generator { run }) = ...
```

i zademonstruj ich użyteczność.

Zadanie 4 (2p). Zaimplementuj metodę `take` dla generatorów, która zwraca listę pierwszych n elementów wygenerowanych przez generator. To zadanie łatwo jest zrobić używając lokalnego mutowalnego stanu. Posłuż się implementacją stanu z wykładu.

Zadanie 5 (6p). Na stronie przedmiotu znajduje się prosta implementacja języka *Brainfuck* napisana we Framie. Implementacja zawiera tylko parser i kilka przydatnych funkcji. Rozszerz ją o interpreter. Zrób to tak, by pokazać, że efekty algebraiczne są składalne. Niech twój interpreter korzysta z trzech niezależnych efektów: taśmy, czytania znaków z wejścia i pisania znaków na wyjście. Następnie doimplementuj odpowiednie handlers tak, by można było uruchomić interpreter.

Zadanie 6 (4p). Obliczenia z lekkimi wątkami można wyrazić przy pomocy następującego efektu.

```
pub data LWT E =  
{ fork : Unit ->[E] Bool  
  , yield : Unit ->[E] Unit  
  , exit : {type X} -> Unit ->[E] X  
}
```

Operacja `fork` tworzy kopię bieżącego wątku, więc wraca dwa razy. Operacja `yield` dobrowolnie oddaje sterowanie innemu wątkowi. Zaimplementuj handler dla tego efektu. W moim rozwiązaniu potrzebowałem rekurencyjnego typu danych reprezentującego wątki (czekające na bieżącą kolejkę wątków)

```
data rec Thr E =  
{ run : Unit ->[E] -> List (Thr E) ->[E] Unit }
```

zaś sam handler przypominał handler stanu, gdzie stanem była kolejka wątków gotowych do wykonania.